1. Design in the right order

1. **Clarify requirements:** users, core actions, read/write ratio, freshness, ordering, privacy, availability, latency, retention, geo-scope.
2. **Estimate scale:** users, requests/s, peak multiplier, object size, storage growth, bandwidth, hot keys.
3. **Define APIs + entities:** stable identifiers, idempotency, pagination, errors, ownership, access control.
4. **Find bottleneck:** CPU, DB, network, disk, lock, queue, hot partition. Scale the bottleneck first.
5. **Choose primitives:** cache, CDN, LB, queue, partitioning, replication, object store, search index.
6. **Close the loop:** failure modes, retries/timeouts, recovery, metrics, logs, traces, SLOs.

Takeaway: A good design is a chain of justified choices: requirement → workload → bottleneck → mechanism → failure/performance behavior.

2. Back-of-the-envelope sizing

$$\text{QPS}_{avg} = \frac{\text{requests in window}}{\text{seconds in window}}; \quad \text{QPS}_{peak} = p\,\text{QPS}_{avg}\ (p \approx 2\text{–}5)$$

$$\text{storage} = \text{objects} \times \text{bytes/object} \times (1 + \text{index/metadata}) \times \text{replicas}$$

$$\text{bandwidth} = \text{QPS} \times \text{average payload}; \quad \text{concurrency} \approx \text{QPS} \times \text{latency}$$

$$\text{cache memory} = \text{hot objects} \times \text{object size}; \quad \text{headroom} \geq 20\text{–}30\%$$

- 1 month $\approx 2.6 \times 10^6$ s; 1 day = 86,400 s.
- 1 billion $\approx 10^9$; 1 TB $\approx 10^{12}$ bytes.
- Replication improves availability/read capacity but multiplies write/storage cost.

Worked: URL shortener

500M new URLs/month, read:write = 100 : 1. Writes ≈ 500M/2.6M = 193 QPS; reads ≈ 19,300 QPS. If 30B URLs over 5 years and 500B/row: raw = 15 TB; with 3× replication ≈ 45 TB, before indexes/overhead. Cache hot redirects; separate read path from write path.

3. Core architecture blocks

Client/CDN	browser cache, static assets, geo-local delivery
LB/proxy	TLS termination, health checks, routing, rate limits
App tier	stateless services; horizontal replicas; API validation
Cache	fast, stale-tolerant reads; TTL/LRU; invalidate carefully
DB	source of truth; indexes, partitioning, replicas, transactions
Queue	async work, buffering, retries, backpressure, decoupling
Object store	large blobs; metadata in DB, content by object ID
Search	derived index for text/analytics; never sole source of truth

Read/write separation (CQRS)

If reads dominate: write to canonical store, asynchronously build read models/materialized views; route reads to replicas/cache. Watch staleness, replication lag, and invalidation.

4. Partitioning, replication, caching

Range	good for scans/time windows; hotspot risk at newest range.
Hash	even distribution/point lookup; poor range scans.
Consistent hash	fewer keys move when nodes change; use vnodes for balance.
Leader–follower	simple ordering; leader is write bottleneck/failure target.
Active–active	scale writes/reads; conflict resolution and consistency become harder.
Cache-aside	read cache; miss reads DB then fills cache; stale data possible.
Write-through	update cache + DB together; simpler reads, slower writes.
CDN	cache geographically close immutable/static content.

Exam cue: Never say "add servers" without explaining what is partitioned, how keys route, how replicas converge, and what happens on a hot key.

5. Design principles

- Modularity limits failure propagation; abstraction separates interface from implementation.
- Layering/hierarchy reduce interactions; naming/indirection enables replacement and late binding.
- Robustness: tolerate expected input variation; fail fast on invalid state.
- Design for iteration: start small, measure, preserve conceptual integrity; avoid the second-system effect.
- Every added feature increases interactions and failure modes. Prefer the simplest design that meets SLOs.

6. System-design answer template

1. Functional + non-functional requirements.
2. Scale assumptions and peak workload.
3. APIs and key data entities.
4. High-level flow and component responsibilities.
5. Storage choice, schema/indexes, partition key.
6. Cache/replica/LB/queue decisions.
7. Bottleneck and capacity calculation.
8. Failure handling, consistency, recovery.
9. Observability and evolution path.

7. Common case: Citizen 360 eligibility

Naive approach

For each application, scan Aadhar, tax, education, employment tables. With N rows/table, repeated scans cost $\Theta(5N)$ per request and do not scale. **Better approach**

Resolve the canonical entity once using Aadhar ID (or a carefully designed identity-resolution index), maintain indexes/materialized facts by Aadhar ID, and precompute eligibility after annual updates.

- **CHECK_ELIGIBILITY:** point/index lookups across required sources: approximately $O(k \log N)$ or $O(k)$ with hash indexes, k = number of sources.
- **NOTIFY_SCHEME:** scan a maintained eligibility index once, $O(N)$, then batch notifications; or query only eligible set K, $O(K)$.
- Handle missing/obsolete fields explicitly; record provenance and consent; make notification idempotent.

Exam cue: State how entities are resolved. "Name + DOB" is not a safe unique key; prefer a canonical ID plus confidence/fallback rules.

8. Load balancing

Round robin	simple; assumes similar servers/jobs and statelessness.
Weighted RR	handles heterogeneous capacity.
Least connections	good for long-lived uneven connections.
Least response time	uses observed latency; can amplify noisy measurements.
Hash/sticky	session affinity; hurts rebalancing and hotspots.
Power of two	sample two random servers, pick shorter/better queue; cheap and effective.
Central queue	avoids idle server while work waits; dispatcher/queue can be bottleneck.

9. Availability and failure checklist

- Availability = $\frac{\text{uptime}}{\text{uptime+downtime}}$; "nines" are a downtime budget, not magic reliability.
- Remove single points of failure: replicas, independent failure domains, health checks, failover.
- Separate **fault** (latent defect), **error** (bad state/signal), **failure** (wrong interface behavior).
- Failure response: detect → contain → mask/recover → report/learn.
- Replicas fail together if correlated: same rack, power, region, software bug, or bad rollout.

Retries are dangerous

Use bounded timeouts, exponential backoff + jitter, retry only safe/idempotent operations, cap attempts, and use circuit breakers. Retries multiply load during overload.

10. Observability

Metrics	numeric time series: rate, errors, latency percentiles, saturation, queue depth.
Logs	event details with request/correlation ID; structured and privacy-aware.
Traces	request path across services; find fan-out and slow spans.
SLI	measured reliability signal, e.g. successful requests under 200 ms.
SLO	target for SLI; alerts should be actionable and tied to error budget.

11. Fast trade-off prompts

- Need range scans? Prefer range partitioning; need uniform point lookups? Prefer hashing.
- Need low latency and tolerate stale reads? Cache/replicas; need strict read-after-write? Route to leader or use session guarantees.
- Need durable async work? Queue with persistence, acknowledgements, replay policy, and idempotent consumers.
- Need large files? Object store + metadata DB, not giant DB rows.
- Need more throughput? Find the saturated resource before scaling any tier.

12. Mini design: newsfeed

- Write tweet/post to durable DB; media to object store; publish eve
- Fan-out-on-write for normal users: precompute follower timelines in store.
- Fan-out-on-read for celebrities/hot users: merge their recent posts a
- Partition by user ID; salt hot users; paginate by cursor (*timestam* offset.
- Cache first page, use CDN for media, replicate reads, and ing/notification async.

Exam cue: The right answer depends on follower skew, read/write ratio and whether the feed must be globally ordered.

13. Mini design: URL shortener

- `POST /shorten`: validate URL, allocate random/base62 ID, store ma expiry.
- `GET /{code}`: cache lookup → DB/replica fallback → redirect; increm ics asynchronously.
- Random IDs avoid sequential enumeration; unique constraint handl
- Expiry via TTL or background cleanup; idempotency key prevent creation on retry.

14. Quantitative design cues

- If one machine handles c QPS at target latency, rough m peak QPS/($c \times$ target utilization).
- If DB is the bottleneck, adding app servers alone changes nothing. work with indexes, caching, read replicas, partitioning, or precomp
- If a queue grows without bound, arrival work > service capacity: a shed/load-level, or reduce work.

15. Exam checklist / common traps

- State assumptions and units; convert minutes/hours to seconds cor
- Distinguish service time S from response time T (queue + service).
- Distinguish average utilization from probability all servers are busy
- Explain why a mechanism works, not just name it.
- For complexity: count scans, index lookups, joins, network calls, and sends.
- For every replica/cache: state freshness, consistency, invalidation, behavior.

1. Queueing notation

λ	arrival rate (jobs/s)
S	service requirement/time; $E[S]$ mean service time
μ	service rate $= 1/E[S]$
ρ	utilization; M/M/1: $\rho = \lambda/\mu$
L, L_q	mean jobs in system / queue
W, W_q	mean time in system / queue
X	throughput (completed jobs/s)
R	resource requirement $= \lambda E[S]$ (server-time units)

stable single server: $\rho = \lambda E[S] < 1$

2. Little's Law and core formulas

$$L = \lambda W; \qquad L_q = \lambda W_q$$

$$\text{M/M/1: } W = \frac{1}{\mu-\lambda} = \frac{E[S]}{1-\rho}; \quad W_q = \frac{\rho}{\mu-\lambda}$$

$$L = \frac{\rho}{1-\rho}; \qquad L_q = \frac{\rho^2}{1-\rho}$$

- Response time W includes waiting + service. As $\rho \to 1$, delay explodes.
- M/M/1 state probability: $P_n = (1-\rho)\rho^n$.
- $P\{n \text{ jobs}\}$ is not $P\{n \text{ waiting}\}$; define the state carefully.

Worked: Tech Cafe

If the cafe is empty 20% of the time, M/M/1 $P_0 = 1-\rho = .2$, so $\rho = .8$. Probability of exactly 2 customers: $P_2 = (.2)(.8)^2 = .128 = 12.8\%$.

3. Multi-server and staffing

$$R = \frac{\lambda}{\mu} = \lambda E[S] \quad \text{(offered load / required servers at 100\% utilization)}$$

$$k > R \text{ for stability}; \qquad k \approx R + \beta\sqrt{R} \quad \text{(square-root staffing)}$$

$$B(k,R) = \frac{R^k/k!}{\sum_{i=0}^{k} R^i/i!} \quad \text{(Erlang-B: M/G/k/k loss probability)}$$

- M/M/k with a queue: use Erlang-C for probability of waiting, then add service time.
- More servers can support high aggregate utilization with low probability that all are busy.
- No queue in M/G/k/k: arrivals are Poisson; Erlang-B is insensitive to service distribution beyond mean.

4. Variability and residual life

$$E[\text{residual wait}] = \frac{E[S^2]}{2E[S]} = \frac{E[S]}{2}\left(1 + C_S^2\right)$$

- $C_S = \sigma_S / E[S]$ is coefficient of variation. Variability increases delay.
- Random observers see long intervals more often (inspection paradox).

Worked: bus stop

Mean inter-arrival $E[S] = 10$ min, standard deviation $= 30$ min. $E[S^2] = 10^2 + 30^2 = 1000$ min^2. Expected wait $= 1000/(2 \cdot 10) = 50$ min.

5. Performance laws

$$\text{Utilization: } U_i = X_i S_i = X D_i; \quad \text{Forced flow: } X_i = V_i X; \quad D_i = V_i S_i$$

$$\text{Bottleneck bound: } X \le \frac{1}{D_{\max}}; \qquad D = \sum_i D_i$$

$$\text{Closed system: } X = \frac{N}{R+Z}; \qquad R = \frac{N}{X} - Z$$

- V_i = visits per completed job; S_i = service time per visit; D_i = service demand.
- Open systems: throughput is driven by arrivals if stable. Closed systems: throughput and response are coupled by population N.
- Improve the bottleneck first at high load; balance demands when possible.

6. Scheduling / dispatching

FCFS	fair/simple; long jobs can delay short jobs.
PS	time-share; preemptible requests get progress.
SRPT	minimizes mean response for known sizes (preemptive); can starve long jobs.
JSQ	send to shortest queue; strong for preemptible/low-variance work.
LWL	send to least remaining work; better when sizes vary.
SITA	separate short/medium/long jobs; protects short jobs under high variance.
RR	good with homogeneous servers and regular/equal jobs.
Random	simple; Poisson splitting gives independent queues.

Exam cue: Policy choice depends on preemption, size variability, fairness, load, and whether job size is known.

7. RAID quick table

0	striping	NB	fastest/capacity; any disk failure loses array
1	mirroring	$NB/2$	strong reads/reliability; capacity cost
4	dedicated parity	$(N-1)B$	parity-disk write bottleneck
5	distributed parity	$(N-1)B$	tolerates one disk; small writes pay read-modify-write

- Full-stripe writes avoid read-modify-write overhead.
- XOR parity: old data + new data + old parity can derive new parity; parity must preserve the XOR invariant.
- RAID is transparent: logical blocks map to physical disks/chunks.

8. LSM, WAL, and C-Store

- **WAL:** append intent durably before updating in-memory/index structures; after crash, replay WAL. Provides durability and recovery.
- **LSM tree:** write to memtable + WAL, flush immutable sorted run, compact runs later. Fast sequential writes; read/space/compaction amplification trade-offs.
- **C-Store:** WOS accepts writes; ROS is compressed/read-optimized. Move/merge tuples from WOS to ROS to bound WOS growth and improve scans. Query compressed data when predicates/encoding permit (e.g., dictionary/RLE operations).

9. Dynamo-style systems

Consistent hash	keys map to a ring; nodes own ranges; vnodes spread load and ease rebalancing.
Replication	preference list stores N replicas; quorum parameters R, W trade latency vs consistency.
Sloppy quorum	write to healthy substitutes when preferred replicas unavailable.
Hinted handoff	substitute stores a hint and forwards update when target returns.
Merkle tree	compare range hashes to find replica divergence efficiently.
Zero-hop DHT	client knows partition map and contacts owner directly.
Eventual consistency	concurrent versions need reconciliation/conflict policy.

quorum intuition: $R + W > N \Rightarrow$ read/write sets intersect (under assumptions)

10. SQL vs NoSQL decision

- SQL: rich queries, joins, constraints, transactions, strong integrity, mature optimization.
- NoSQL/document/key-value: flexible schema, horizontal scale, access-pattern-first design, denormalization.
- Choose by access patterns and consistency needs, not by "big data" branding. Financial integrity usually favors transactional/strongly consistent design.

11. Fault-tolerance patterns

Detection	checksums, timeouts, health probes, invariants, quorum.
Containment	modules, bulkheads, isolation, process/rack/region bou
Masking	retries, replicas, erasure/parity, failover, correction.
Fail-fast	reject/report invalid state early.
Fail-stop	stop rather than emit corrupt output; supervisor restar
Fail-soft	continue useful partial service.
Temporal redundancy	retry same component after transient fault.
Spatial redundancy	use independent components/replicas.

$$\text{series availability} = \prod_i A_i; \qquad \text{independent parallel availabilit} \quad 1 - \prod_i (1 - A_i)$$

- Independence is an assumption; correlated failures invalidate the sim
- Maintain an explicit fault model: identify faults → estimate risk → contain → mask → observe/postmortem.

12. Resilient request path

1. Deadline/timeout on every network hop.
2. Retry only transient errors and idempotent operations; exponentia jitter.
3. Circuit breaker after repeated failures; recover gradually.
4. Bulkhead resources by tenant/function; prevent one dependency fro threads.
5. Load shed or rate-limit before queues become unbounded; use load bursts.
6. Make handlers idempotent using request IDs/deduplication keys.

Availability budget

99%	14.4 min/day downtime
99.9%	1.44 min/day
99.99%	8.64 s/day
99.999%	0.864 s/day

13. Mini numerical examples

M/M/1

$\lambda = 8$/s, $\mu = 10$/s: $\rho = .8$, $W = 1/(10-8) = 0.5$s, $L = \lambda W$ $W_q = W - 1/\mu = .4$s.

Resource/staffing

$\lambda = 10$/s, $E[S] = 0.2$s: $R = 2$ server-seconds/s. At 70% target utiliza capacity $k \ge 2/.7 = 2.86$, so choose at least 3 servers before headro requirements.

Availability

Two independent replicas each at $A = .99$: parallel service availability 1 .9999. If both share one power supply, independence is false; fix the failure domain.

14. One-minute exam response

1. State assumptions, units, workload, and target metric.
2. Write the governing law/formula before plugging numbers.
3. Show bottleneck/stability check.
4. For architecture: name components, data flow, partition key, repl failure path.
5. Finish with trade-off and one monitoring signal.

Exam cue: Partial credit comes from explicit assumptions, correct justification. A named technology without a workload/consistency a not a design.

15. Formula sanity checks

- $\rho \ge 1$ for a single-server queue $\Rightarrow$ no finite steady-state mean delay
- $L = \lambda W$ must have consistent units: jobs = jobs/s $\times$ s.
- More replication never removes write coordination or correlated failu
- Caching lowers DB load only if hit rate, freshness, and invalidation ar
- Averages hide tail latency: inspect p95/p99 and queue depth, not o